

Intern Problem Statement

Real-Time FIR Filtering of a UART Data Stream on an iCE40 FPGA | Duration: 4 weeks | Mentor: Ajith Peter

1. Objective

Design, simulate, and implement a **Finite Impulse Response (FIR) digital filter** in Verilog on the Lattice iCE40 development board. The system shall receive a stream of data samples over a **UART receive (RX)** line, apply FIR filtering using a fixed set of coefficients hard-coded into the design, and transmit the filtered samples out over a **UART transmit (TX)** line in real time.

2. Background

An FIR filter computes each output as a weighted sum of the current and past $N-1$ input samples: $y[n] = \sum b_k \cdot x[n-k]$, where b_k are the filter coefficients (taps). Unlike IIR filters, FIR filters are unconditionally stable and can have exactly linear phase, which makes them a staple of FPGA-based signal processing. In hardware, the filter maps naturally onto a shift register (delay line), an array of multipliers, and an adder tree — an excellent first exercise in datapath design, fixed-point arithmetic, and clock-domain-aware I/O.

3. Functional Requirements

- R1 – UART RX:** Implement a UART receiver (8N1, suggested 115200 baud) that deserializes incoming bytes; each byte is one signed 8-bit sample.
- R2 – FIR core:** Implement an N -tap FIR filter ($N = 8-16$, e.g. a low-pass design) with signed fixed-point coefficients stored as Verilog parameters / localparams — no run-time coefficient loading required.
- R3 – Arithmetic:** Use fixed-point multiply-accumulate with correct bit-growth; scale/truncate (round) the accumulator back to 8 bits for output, with saturation on overflow.
- R4 – UART TX:** Implement a UART transmitter that serializes each filtered sample back out; the pipeline must sustain the full incoming data rate without dropping samples.
- R5 – Verification:** Provide a self-checking Verilog testbench that feeds known inputs (impulse, step, and a noisy sine) and compares the RTL output against a golden model computed in Python/NumPy (scipy.signal).

4. Tools and Constraints

Use the open-source iCE40 flow: **Yosys** (synthesis), **nextpnr-ice40** (place & route), **IceStorm / iceprog** (bitstream & programming), and **Icarus Verilog + GTKWave** (simulation). Coefficients should be designed in **Python (scipy.signal.firwin)** and quantized to fixed point. Write clean, synchronous, synthesizable Verilog-2001; a single clock domain derived from the on-board oscillator; all modules commented and kept in a Git repository from day one. A USB-UART adapter (e.g. FTDI) connects the PC to the board's RX/TX pins.

5. Execution Roadmap (4 Weeks)

Phase	Activities and Milestones
Week 1 Study & Design	Revise FIR theory (convolution, taps, frequency response, fixed-point quantization). Design coefficients in Python, plot the frequency response, and quantize them. Study the UART frame format and the iCE40 open-source flow; run a blink-LED build to validate the toolchain. Milestone: one-page design note with block diagram, chosen coefficients, and bit-widths.
Week 2 FIR Core in RTL	Code the FIR datapath (shift register, MAC, scaling/saturation) with a valid-strobe interface. Simulate with Icarus Verilog; verify impulse response equals the coefficient list and outputs match the NumPy golden model bit-exactly. Milestone: self-checking testbench passing all vectors.
Week 3 UART & Integration	Code and simulate UART RX and TX modules (baud-rate generator, start-bit detection, majority sampling). Integrate RX → FIR → TX with proper handshaking; simulate the full chain end-to-end. Milestone: full-system simulation streaming bytes through the filter.
Week 4 Hardware & Demo	Synthesize with Yosys, place-and-route with nextpnr, review timing and resource usage, and program the board. Write a Python host script that sends a noisy waveform over the serial port, captures the filtered return, and plots input vs. output. Milestone: live demo plus final report.

6. Deliverables

(1) Git repository with all Verilog sources, testbenches, constraint file, and build scripts; (2) Python scripts for coefficient design and the host-side send/capture/plot demo; (3) simulation waveforms and golden-model comparison results; (4) a short final report (≤ 5 pages) covering the architecture, fixed-point choices, resource utilization, maximum clock frequency, and lessons learned; and (5) a live demonstration on the iCE40 board.

7. Evaluation Criteria

Correctness of the filter output against the golden model (40%); quality and readability of the RTL and testbenches (25%); depth of understanding shown in the report and design reviews (20%); and the working hardware demonstration (15%). Weekly check-ins with the mentor are expected; asking questions early is encouraged — the goal is understanding, not just a working bitstream.